

Infosys Springboard Virtual Internship 7.0 Completion Report

Team Details

Batch Number-02

Start date-15th July 2026

Names: Puspall Das | Mittapalli Praharsha | Muthyala Meghana | Ramanaboina Sowjanya

Internship Duration: 8 Weeks

1. Project Title

Intelligent Farm Operations Platform with Smart Agro Precision System

2. Project Objective

The primary goal of the **FarmVerse** project is to digitize and optimize traditional agricultural practices by developing a unified, Java Full-Stack Precision Agriculture Management Platform.

The project aimed to achieve this by centralizing critical farming operations into a single ecosystem. It successfully integrates a gamified farmer dashboard to drive daily digital engagement, a secure digital marketplace that eliminates traditional middlemen to ensure fair pricing, and a real-time Generative AI advisory system (powered by Google Gemini) that provides instant, data-driven solutions for crop and pest management.

Its relevance to the organization (and the broader agricultural sector) lies in bridging the technological gap for rural communities. By transforming fragmented, manual farming methods into a secure, cloud-based workflow utilizing modern technologies (Spring Boot, MongoDB, and AI), FarmVerse significantly empowers farmers to make informed decisions, optimize their crop yields, and maximize their economic returns.

3. Project description in detail

1. Project Overview FarmVerse is a robust, Java Full-Stack Precision Agriculture Management Platform designed to modernize and streamline traditional farming workflows. The project serves as a centralized digital ecosystem that bridges the technology gap for farmers. It successfully consolidates agricultural advisory, crop lifecycle management, and a secure farmer-to-buyer digital marketplace into a single, highly accessible web application. By shifting farmers away from fragmented, manual processes, FarmVerse provides a data-driven environment that optimizes farm productivity and profitability.

2. Approach and Methodology The development of FarmVerse followed an agile, modular approach, separating the architecture into three distinct layers to ensure security, scalability, and performance:

- **User-Centric Design (Frontend):** Recognizing that technology adoption can be low in rural sectors, the frontend was designed to be lightweight, responsive, and engaging. A "Gamification Engine" was implemented, utilizing Experience Points (XP) and daily missions to encourage consistent user engagement.

- **Secure API Architecture (Backend):** The backend was structured as a secure RESTful API. JSON Web Tokens (JWT) and Spring Security were implemented to enforce strict Role-Based Access Control (RBAC), ensuring that Farmers, Administrators, and Agricultural Experts only have access to their authorized data and modules.
- **AI Integration:** Instead of relying on static, hardcoded advisory systems, an innovative approach was taken to integrate real-time Generative AI. The system communicates securely with Google's Gemini LLM to provide context-aware agricultural advice. To ensure enterprise reliability, custom retry logic was engineered into the backend to handle high server demand and network latency gracefully.

3. Technologies Used

The project was built using a modern, scalable technology stack:

- **Frontend:** HTML5, CSS3, and Vanilla JavaScript (Ensuring ultra-fast load times without heavy framework overhead).
- **Backend:** Java 21, Spring Boot 3.x, Spring Web, and Spring Security.
- **Database:** MongoDB Atlas (A Cloud NoSQL database chosen for its flexibility in storing unstructured AI chat histories and complex marketplace metadata).
- **AI Engine:** Google Gemini 3.6 Flash API (Integrated via Spring RestTemplate).
- **Build & Deployment:** Maven.

4. Real-World Impact and Implementation

In a real-world implementation, FarmVerse carries immense socio-economic impact for the agricultural sector:

- **Eradicating Middlemen:** The integrated Digital Marketplace allows farmers to list their harvests and sell directly to buyers. This transparent commerce model eliminates exploitative third-party fees, significantly increasing the farmer's profit margins.
- **Instant Expert Accessibility:** Traditional farming often suffers from delayed disease diagnosis or poor fertilizer planning due to a lack of available agronomists. FarmVerse's AI Advisor acts as a 24/7 agricultural expert in the farmer's pocket, instantly resolving complex queries regarding pest control and weather delays, ultimately saving crops from failure.
- **Data-Driven Farming:** By utilizing the Crop Sowing Calendar and keeping an audited history of AI advisory, farmers transition from guesswork to calculated, historical data-driven planning, leading to optimized yields and sustainable farming practices.

4. Timeline Overview

Week	Activities Planned	Activities Completed
Week 1	Pre plan of the website , ppt	All
Week 2	Landing page , dashboard	All
Week 3	Database connection	All
Week 4	User profile, ui/ux design, gamify	All
Week 5	Marketplace, crop calendar	All
Week 6	AI integration like chatbot etc.	All
Week 7	Admin front Back	All
Week 8	Deployment, review	All

5a. Key Milestones

Milestone	Description	Date Achieved
Project Kickoff	Project allocated	July 15, 2026
Prototype/First Draft	Presentation of ppt + project	July 21, 2026
Mid-Term Review		
Final Submission		
Presentation		September 20, 2026

5b. Project execution details

The execution of the **FarmVerse** project followed a structured Agile Software Development Life Cycle (SDLC) to ensure continuous improvement, modularity, and timely delivery. The execution was broken down into the following key phases:

1. Requirement Gathering & Feasibility Analysis The project was initiated by identifying the core challenges faced by modern farmers—namely the lack of direct market access, delayed pest identification, and low engagement with digital tools. Based on these problems, the core requirements were finalized: a secure authentication system, a gamified dashboard, a digital marketplace, and an AI-powered advisory system. A feasibility study confirmed the use of Java (Spring Boot) for enterprise-grade backend security and MongoDB for flexible, scalable data storage.

2. System Architecture & Database Design Before writing code, the system architecture was laid out. A three-tier architecture (Client-Server-Database) was designed. The MongoDB collections were modeled to handle unstructured data, such as dynamic AI conversation histories and flexible marketplace listings. Secure Role-Based Access Control (RBAC) was mapped out to ensure logical separation between the Farmer, Admin, and Expert workflows.

3. Backend Development & API Construction Execution of the backend began by setting up the Spring Boot environment using Maven.

- **Security Implementation:** Spring Security was configured to generate and validate JSON Web Tokens (JWT) for stateless user sessions.
- **Core Modules:** RESTful APIs were developed for user authentication (AuthController), gamification tracking (UserController), and marketplace listings (MarketplaceController).
- **AI Integration:** A dedicated AIController was engineered to handle HTTP requests via Spring's RestTemplate. It was securely connected to the Google Gemini 3.6 Flash API. Custom error-handling and retry logic (for HTTP 503 errors) were executed here to ensure the AI assistant remained highly reliable under load.

4. Frontend UI/UX Development The frontend was executed using native HTML5, CSS3, and Vanilla JavaScript to guarantee high performance and low latency.

- A dynamic, gamified **Farmer Dashboard** was built to display daily missions, Experience Points (XP), and usage analytics.
- Asynchronous JavaScript (fetch API) was implemented to seamlessly connect the user interface to the Spring Boot backend, allowing farmers to chat with the AI and post marketplace listings without page reloads.

5. System Integration & Testing Once both layers were developed, the frontend and backend were integrated.

- **Functional Testing:** Each module (Crop Calendar, AI Chatbot, Marketplace) was manually tested to ensure data flowed correctly to and from MongoDB.
- **Security Testing:** API endpoints were tested to ensure that users without valid JWTs were strictly blocked from accessing admin or private farmer data.
- **Edge-Case Handling:** The AI chatbot was tested under simulated heavy loads to verify that the retry logic successfully caught API timeouts.

6. Final Review & Documentation The final phase of execution involved code refactoring, removing unused dependencies, and generating comprehensive documentation (including the README.md and Project_Report.html). Environment variables (such as database passwords and Gemini API keys) were securely abstracted from the source code to ensure the platform was production-ready and safe for demonstration

6. Snapshots / Screenshots

To Run the Project locally the commands are-

```
$env:GEMINI_API_KEY="AQ.Ab8RN6LSoksqcqWZA-a6mgBlaT7Ou17mT7x0534OzNA2OjOA";
```

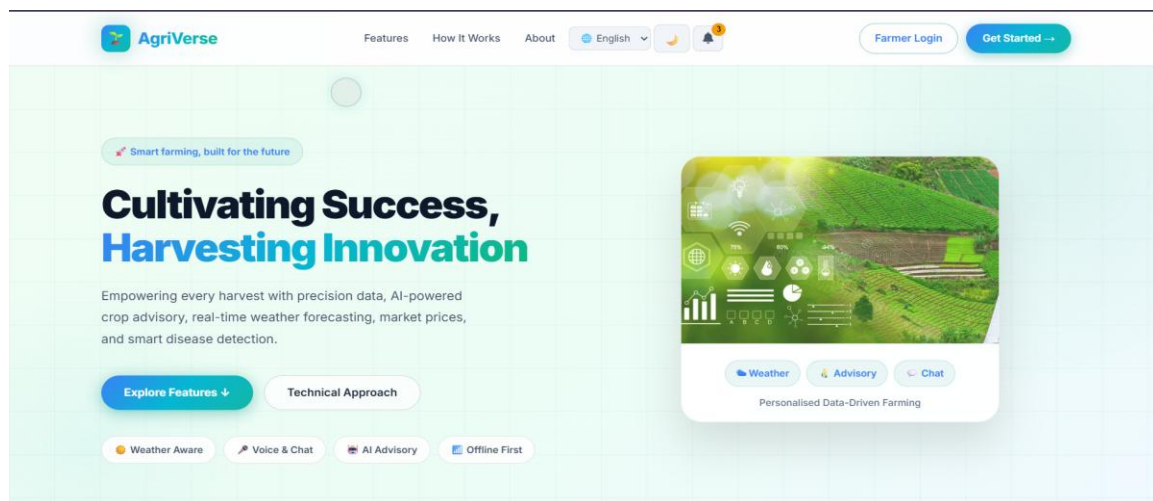
```
$env:MONGO_PASSWORD="agrinfo27";
```

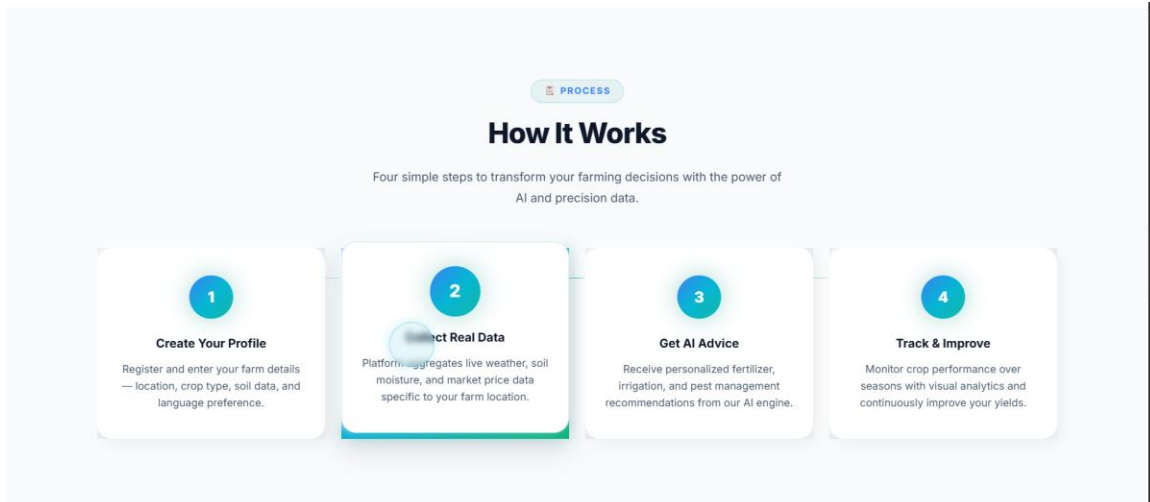
```
mvn spring-boot:run
```

Snippet 1 Architectural Design Snapshot

```
graph TD
    Client[Frontend: HTML/CSS/JS] -->|REST API Requests + JWT| Gateway[Spring Boot REST Controllers]
    subgraph Spring_Boot_Backend [Spring Boot Backend]
        Gateway --> Auth[Spring Security / JWT Auth]
        Auth --> UserService[Gamification & XP]
        Auth --> MarketService[Digital Marketplace]
        Auth --> AIService[AI Controller]
        AIService -->|HTTP POST| Gemini[Google Gemini 3.6 Flash]
    end
    end
    UserService -->|Spring Data| DB1[(MongoDB Atlas Cloud)]
    MarketService -->|Spring Data| DB2[(MongoDB Atlas Cloud)]
    AIService -->|Log History| DB3[(MongoDB Atlas Cloud)]
```

Snippet 2 Website landing page





Offline First
Server-side worker caching ensures core functionality works even in low-connectivity rural areas.

Disease Detection
CNN model trained on PlantVillage dataset — upload a leaf image for instant AI diagnosis.

Privacy First
JWT authentication, bcrypt password hashing, and MongoDB Atlas encryption keep farmer data safe.

AgriVerse
Smart farming powered by Artificial Intelligence, precision weather intelligence, and data-driven agriculture.

© 2026 AgriVerse. All Rights Reserved. Built with ❤️ for Indian Farmers.

Snippet 3 Secure authentication interfaces featuring role-based login portals for both farmers and administrators

AgriVerse
Join 10,000+ Smart Farmers

Create Account

Already have an account? [Sign in instead](#) →

First Name: Last Name:

Email Address:

Phone Number: Village / City:

Gender: ☒ Male ☐ Female ☐ Other

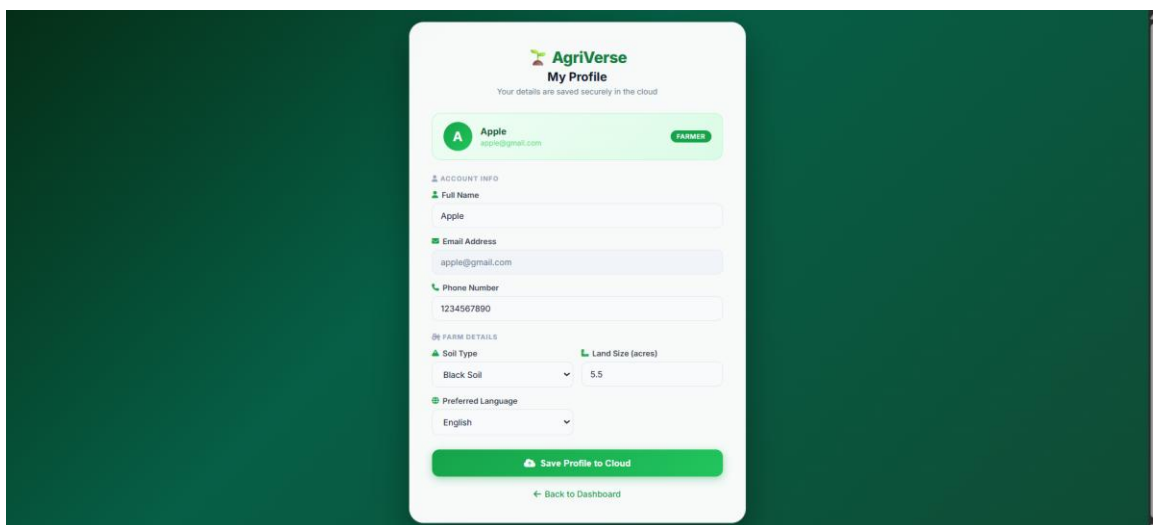
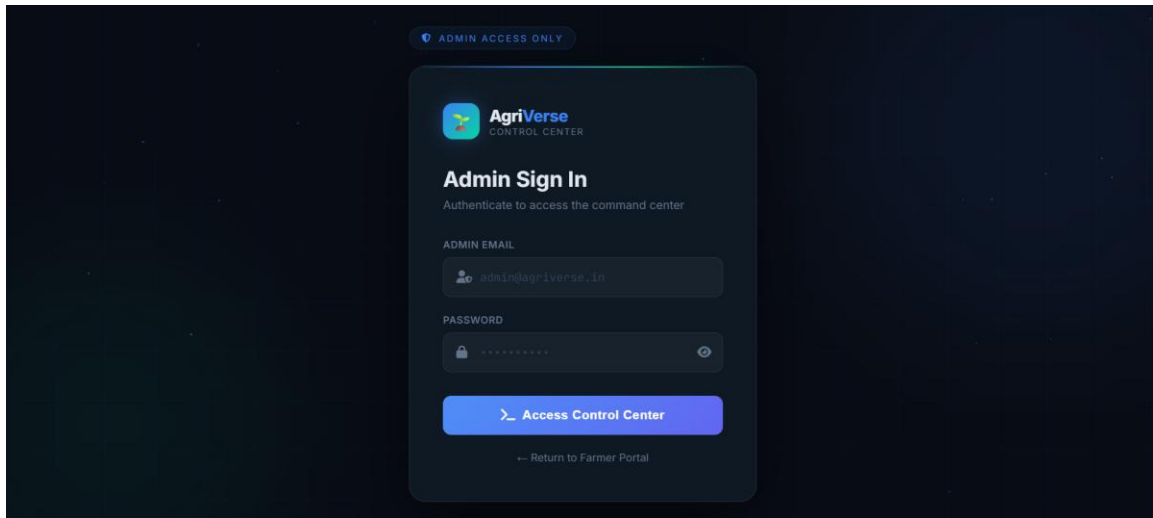
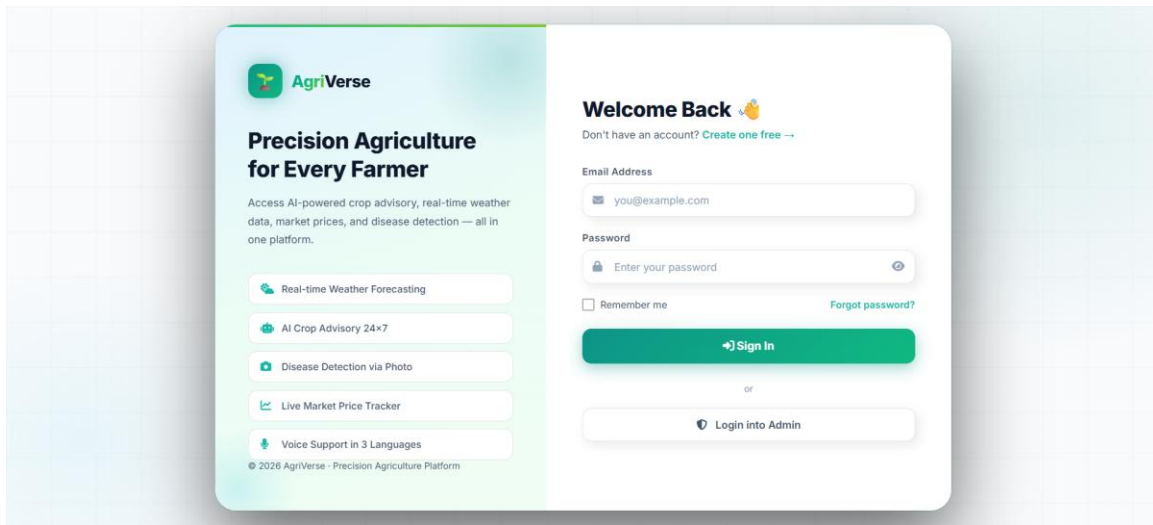
Password: Confirm Password:

☐ I agree to the [Terms of Service](#) and [Privacy Policy](#). I understand that my farm data will be used to provide personalized AI recommendations.

[Create Free Account](#) →

or

Already a member? [Sign in to your account](#) →



Snippet 4 A highly interactive user dashboard designed to drive farmer engagement through gamification, achievements, and regional leaderboards

The screenshot displays the AgriVerse app interface for a user named Apple. The top navigation bar includes links to Dashboard, Leaderboard, Marketplace, and Calendar, along with language and user settings. The user profile section shows Apple at Level 1 with a 4-day streak and 275 XP, with a progress bar for Level 2. User details include Phone (1234567890), Soil Type (Black Soil), Land Size (5.5 acres), and Language (English). The Subscription & Usage section indicates the user is on a 'Free Plan' with 'Active Free' status and 'Current Plan 2026'. Below this, a row of tabs includes Daily Missions, AI Assistant, Pest Detector, Advisory History, Usage Stats, and Analytics. The 'Today's Farming Missions' section lists several tasks, all marked as 'Completed' (Compl. Mission): Irrigate Paddy Field (+50 XP), Inspect Tomato Leaves (+20 XP), Apply NPK Fertilizer (+30 XP), Log Weather Observation (+15 XP), Clean Drip Lines (+25 XP), Check Pheromone Traps (+40 XP), Record Soil Moisture (+35 XP), and Complete All Daily Tasks (+60 XP BONUS). A 'Farmer Achievements' section shows icons for First Harvest, Soil Master, Pest Hunter, and AI Pioneer. On the right, the 'Village Leaderboard' lists top ranked precision farmers, with Apple (You) at #7 with 275 XP. A 'Predictive Pest Alerts' section shows a 'High Risk' alert for Brown Planthopper in Rice/Paddy and a 'Medium Risk' alert for Whitefly / Leaf Curl Virus in Tomato. The footer includes copyright information for AgriVerse and links to References and Benefits.

AgriVerse

Dashboard Leaderboard Marketplace Calendar

English

Apple

Apple

Lvl 1 4 Day Streak 275 XP

Progress to Level 2 275 / 100 XP

PHONE 1234567890

SOIL TYPE Black Soil

LAND SIZE 5.5 acres

LANGUAGE English

Profile

Subscription & Usage

Free Plan

Plan Status Active Free

Current Plan 2026

Upgrade to Premium

Daily Missions AI Assistant Pest Detector Advisory History Usage Stats Analytics

Today's Farming Missions Resets in 14h 22m

Irrigate Paddy Field Compl. Mission +50 XP

Inspect Tomato Leaves Compl. Mission +20 XP

Apply NPK Fertilizer Compl. Mission +30 XP

Log Weather Observation Compl. Mission +15 XP

Clean Drip Lines Compl. Mission +25 XP

Check Pheromone Traps Compl. Mission +40 XP

Record Soil Moisture Compl. Mission +35 XP

Complete All Daily Tasks Cla Bonus +60 XP BONUS

Farmer Achievements

First Harvest Soil Master Pest Hunter AI Pioneer

Village Leaderboard Top ranked precision farmers in your sector

Rajesh Kumar 1250 XP

Lakshmi Narayana 980 XP

Suresh Reddy 845 XP

#4 Anita Desai 620 XP

#5 Vikram Singh 315 XP

#6 Administrator 315 XP

#7 Apple (You) 275 XP

#8 Priya Sharma 110 XP

#9 Info Sys 110 XP

#10 Farmer User 40 XP

#11 Mrinmay Das 40 XP

View Full Leaderboard

Predictive Pest Alerts HIGH ALERT

48-Hour Outbreak Radar

Rice / Paddy High Risk

Brown Planthopper

High risk vector detected due to recent humidity spike.

Spray Neem oil 10,000 PPM @ 3m/litre.

Tomato Medium Risk

Whitefly / Leaf Curl Virus

Medium vector threat observed in neighboring plots.

Install yellow sticky traps @ 10/acre.

High Risk Medium Risk Low Risk

© 2026 AgriVerse. All rights reserved. References Benefits

Snippet 5 Integrated AI capabilities and analytics, featuring a real-time Generative AI chat, computer-vision pest detection, advisory history logging, and farm analytics

Daily MissionsAI AssistantPest DetectorAdvisory HistoryUsage StatsAnalytics

AgriVerse AI Agricultural Advisor

Ask questions about crop diseases, fertilizers, and weather advisory.

Hello! I am your AgriVerse AI Assistant. How can I help your farm today?

What fertilizer is best for paddy?

Hello! Welcome to FarmVerse. For paddy (rice), there isn't a "magic" fertilizer. Instead, paddy thrives best on a **balanced combination of NPK (Nitrogen, Phosphorus, Potassium) applied in splits, plus Zinc.** Here is the most practical and proven fertilizer schedule for paddy: ### 1. Basal Dose (At Sowing/Transplanting) * **What to use:** **DAP (Di-Ammonium Phosphate)** or **NPK Complex (like 10-26-26 or 12-32-16)** * **MOP (Muriate of Potash)** * **Add Micronutrient:** Apply **Zinc Sulfate (10 kg/acre)** at land preparation. * Zinc is crucial for paddy to prevent leaf browning (Khaira disease). * ### 2. First Top-Dressing (Active Tillering Stage: 20-25 days after transplanting) * **What to use:** **Urea** (Neem-Coated Urea is best). * **Tip:** Keep the field moist, but drain standing water slightly before applying Urea for maximum absorption. * ### 3. Second Top-Dressing (Panicle Initiation Stage: 40-45 days after transplanting) * **What to use:** **Urea + MOP (Potash)** * Ask AI about crop disease, soil, or irrigation...

What fertilizer is best for paddy?How to control tomato blight?Best irrigation time tomorrow?

Daily MissionsAI AssistantPest DetectorAdvisory HistoryUsage StatsAnalytics

AI Crop Pest & Disease Diagnostic Scanner

Upload a clear photo of your crop leaf or stem for instant AI diagnosis.

Click or drag & drop leaf photo here to scan

Supports JPG, PNG, WEBP up to 10MB

Upload Pest Image

Fall Armyworm (*Spodoptera frugiperda*) Detected

Confidence: 86.3%

Significant leaf damage and ragged holes observed. Larvae presence likely in the whorl.

Recommended Action:

Apply Spinetoram 11.7% SC @ 0.5ml/litre or Chlorantraniliprole 18.5% SC @ 0.4ml/litre during cool hours.

Daily MissionsAI AssistantPest DetectorAdvisory HistoryUsage StatsAnalytics

Resource & Quota Usage

Current usage meters for your free tier quota

Pest Scans Used

5 / 10 Scans Used

AI Chat Queries

14 / 20 Queries Used

Weather Radar Alerts

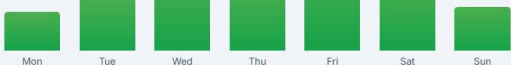
Unlimited Active

Daily MissionsAI AssistantPest DetectorAdvisory HistoryUsage StatsAnalytics

Farm Analytics & Outbreak Trends

Historical disease risk and soil moisture trends

Pest Outbreak Risk Trend (7 Days)



Day	Risk Level
Mon	Low
Tue	Low
Wed	Medium
Thu	Medium
Fri	Medium
Sat	Low
Sun	Low

Soil Moisture & Temperature Log



Soil Moisture: 38%Temp: 29°C

pg. 8

Daily Missions	AI Assistant	Pest Detector	Advisory History	Usage Stats	Analytics
Advisory History Complete history of your AI Assistant and Pest Detector activities. Refresh History					
Date	Type	Crop	Issue / Query	Recommendation / Response	
20 Sept 2026 10:43 am	AI Assistant	Rice	What fertilizer is best for paddy?	Hello! Welcome to FarmVerse. For paddy (rice), there isn't a single "magic" fertilizer. Instead, paddy thrives best on a **balanced combination of NPK (Nitrogen, Phosphorus, Potassium) applied in splits, plus Zinc.** Here is the most practical and proven fertilizer schedule for paddy: ### 1. Basal Dose (At Sowing/Transplanting) * **What to use:** **DAP (Di-Ammonium Phosphate)** or **NPK Complex (like 10-26-26 or 12-32-16)** + **MOP (Muriate of Potash)** + **Add Micronutrient:** Apply **Zinc Sulfate (10 kg/acre)** at land preparation. * Zinc is crucial for paddy to prevent leaf browning (Khaira disease). * ### 2. First Top-Dressing (Active Tilling Stage: 20-25 days after transplanting) * **What to use:** **Urea** (Neem-Coated Urea is best). * *Tip:* Keep the field moist, but drain standing water slightly before applying Urea for maximum absorption. ### 3. Second Top-Dressing (Panicle Initiation Stage: 40-45 days after transplanting). * **What to use:** **Urea + MOP (Potash)** + *Why:* Potassium at this stage helps in grain filling, prevents lodging (falling over), and improves pest resistance. --- ### Quick Pro-Tips for FarmVerse Farmers: * *Organic Boost:* Incorporate **Green Manure**.	

Snippet 6 Direct market integration and visual crop planning modules, designed to optimize farmer profitability and harvest scheduling

AgriVerse

DashboardAboutEnglishAP

The Farmer's Marketplace

Discover fresh, direct farm-to-consumer produce across India. Connect directly with farmers and buy high-quality organic and conventional crops.

[Refresh Data](#)[+ Post New Listing](#)

All States

All Categories

GrainVegetableFruitSpicePulseOther

4 listings found

2 organic

ORGANIC

Organic Tomatoes (100kg)

Ravi Kumar

PRICE

₹2500 per kg

AVAILABLE

100 kg

Crop

28 days ago

Freshly harvested organic tomatoes from North India.

Contact Seller

Used Tractor - Massey Ferguson

Sharma Rao

PRICE

₹350000 per piece

AVAILABLE

1 piece

Equipment

28 days ago

2018 model, good condition.

Contact Seller

Premium Urea Fertilizer

Lakshmi Narayana

PRICE

₹800 per bags

AVAILABLE

5 bags

Fertilizer

28 days ago

Surplus from last season. Sealed bags.

Contact Seller

Create New Listing

Crop Name

e.g. Tomatoes

Category

Vegetable

Price (with unit)

e.g. ₹2000 per quintal

Location (City, State)

e.g. Nashik, Maharashtra

Quantity Available

e.g. 500 kg

☐ Certified Organic Product

Post Listing

Crop Sowing Calendar

Discover the optimal sowing and harvest windows for your crops based on seasonal requirements.

[Suggest a Crop](#)

Select Month

Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec

Weather Needs

At Low Medium High

Showing crops for Jan

Tomato

RABI SEASON

SOW: Jan - Jan
HARVEST: Apr - May

WATER: Medium
SOIL: Loamy

Tomato is a short-duration vegetable crop suitable for cultivation in the Rabi season.

Chickpea

RABI SEASON

SOW: Sep - Oct
HARVEST: Jan - Feb

WATER: Low
SOIL: Loamy

Chickpea is an important pulse crop mainly cultivated during the Rabi season.

QWE

RABI SEASON

SOW: Jan - Feb
HARVEST: Mar - Apr

WATER: High
SOIL: BLACK

AA

■ Sowing Window ■ Harvest Window

Suggest a Crop

Your suggestion will be reviewed by an administrator.

Crop Name (e.g. Wheat)

Season (e.g. Rabi)

Sowing Start

Sowing End

Harvest Start

Harvest End

Water Need

Soil Type (e.g. Loamy)

Description & Tips...

Cancel [Submit Suggestion](#)

Snippet 7 The secure Admin Command Center, providing centralized oversight for user management, marketplace moderation, dispute resolution, and system analytics.

AgriVerse
COMMAND CENTER

OVERVIEW

Dashboard

User Management

PLATFORM

Marketplace Moderation

Disputes

Crop Calendar

Live Activity Feed

System Health

admin@agriverse.in
SUPER ADMIN

Logout

AGRIVERSE ADMIN
OVERVIEW DASHBOARD

Quick search...

System Online

13
Total Users
Live from database

10
Farmers
Active role

15
Active Now
Approximate

3
Admins
Privileged accounts

Recent Registrations

USER	ROLE	ACTIONS
Mrimay Das mrd@gmail.com	USER	View All
pavan m pavan@gmail.com	USER	View All
Raju m raju@gmail.com	USER	View All
Mitapali Praharsha 23rdsab@agriverse.edu.in	USER	View All
Mitapali Praharsha mitapaliv@agriversemail.com	USER	View All

Live Activity

- New farmer registered: Ravi Kumar (2 min ago)
- Admin signed in from Hyderabad, IN (5 min ago)
- Crop listing Organic Tomatoes submitted (12 min ago)
- Security: JWT refreshed for priya@gmail.com (16 min ago)
- AI advisor queried for cotton disease detection (24 min ago)

AgriVerse
COMMAND CENTER

OVERVIEW

Dashboard

User Management

PLATFORM

Marketplace Moderation

Disputes

Crop Calendar

Live Activity Feed

System Health

admin@agriverse.in
SUPER ADMIN

Logout

AGRIVERSE ADMIN
MARKETPLACE

Quick search...

System Online

Marketplace Listings

9 total listings

Search by title

All Pending Approved Rejected

TITLE	CATEGORY	PRICE	FARMER	STATUS	ACTIONS
Organic Tomatoes (100kg)	Crop	₹2500/kg	Ravi Kumar	APPROVED	👍👎
Used Tractor - Massey Ferguson	Equipment	₹350000/piece	Sneha Rao	APPROVED	👍👎
Premium Urea Fertilizer	Fertilizer	₹800/bags	Lakshmi Narayana	APPROVED	👍👎
AAA	Vegetable	₹20000/kg	Anonymous Farmer	REJECTED	👍👎
brinjal	Vegetable	₹300/kg	Anonymous Farmer	PENDING	👍👎
aqw	Fruit	₹11/kg	Anonymous Farmer	REJECTED	👍👎
Tomato	Vegetable	₹200/kg	Anonymous Farmer	APPROVED	👍👎
QWE	Fruit	₹200/kg	Anonymous Farmer	REJECTED	👍👎
potatos	Vegetable	₹80/kg	Anonymous Farmer	PENDING	👍👎

AgriVerse COMMAND CENTER

OVERVIEW

- Dashboard
- User Management
- PLATFORM
- Marketplace Moderation
- Disputes**
- Crop Calendar
- Live Activity Feed
- System Health

admin@agriverse.in
SUPER ADMIN

Logout

AGRIVERSE ADMIN
DISPUTES

Quick search...

System Online

Dispute Resolution 3 total disputes

All Open Resolved

ID	BUYER	SELLER	REASON	STATUS	ACTIONS
405458	Amit Singh	Ravi Kumar	Tomatoes arrived spoiled due to poor packa...	RESOLVED	 
405459	Priya Sharma	Sneha Rao	Tractor battery was completely dead on deliv...	RESOLVED	 
405460	a	Ravi Kumar	product is not good	RESOLVED	 

AgriVerse COMMAND CENTER

OVERVIEW

- Dashboard
- User Management
- PLATFORM
- Marketplace Moderation
- Disputes
- Crop Calendar**
- Live Activity Feed
- System Health

admin@agriverse.in
SUPER ADMIN

Logout

AGRIVERSE ADMIN
CROP CALENDAR MANAGEMENT

Quick search...

System Online

Crop Sowing Calendar + Add New Crop

CROP NAME	SEASON	SOWING WINDOW	HARVEST WINDOW	STATUS	ACTIONS
Rice	KHARIF	Jun - Jul	Oct - Nov	APPROVED	 
Maize	KHARIF	Jun - Jul	Sep - Oct	APPROVED	 
Mango	KHARIF	Aug - Aug	Mar - May	APPROVED	 
Tomato	RABI	Jan - Jan	Apr - May	APPROVED	 
Cucumber	RABI	Feb - Feb	Apr - May	APPROVED	 
Okra	SUMMER	Mar - Mar	May - Jun	APPROVED	 
Groundnut	SUMMER	Apr - Apr	Jul - Aug	APPROVED	 
Cotton	KHARIF	Jul - Jul	Nov - Dec	APPROVED	 
Chickpea	RABI	Sep - Oct	Jan - Feb	APPROVED	 

AgriVerse COMMAND CENTER

OVERVIEW

- Dashboard
- User Management
- PLATFORM
- Marketplace Moderation
- Disputes
- Crop Calendar
- Live Activity Feed**
- System Health

admin@agriverse.in
SUPER ADMIN

Logout

AGRIVERSE ADMIN
ACTIVITY FEED

Quick search...

System Online

Platform Activity Feed Refresh

- New farmer registered: Ravi Kumar 2 hrs ago
- Admin signed in from Hyderabad, IN 3 min ago
- Crop listing Organic Tomatoes submitted 12 min ago
- Security: JWT refreshed for priya@gmail.com 10 min ago
- AI advisor queried for cotton disease detection 24 min ago
- New farmer registered: Sneha Rao 31 min ago
- Failed login attempt for unknown@mail.com 40 min ago
- Database backup completed successfully 1 hr ago

AgriVerse COMMAND CENTER

OVERVIEW

- Dashboard
- User Management
- PLATFORM
- Marketplace Moderation
- Disputes
- Crop Calendar
- Live Activity Feed
- System Health**

admin@agriverse.in
SUPER ADMIN

Logout

AGRIVERSE ADMIN
SYSTEM HEALTH

Quick search...

System Online

Atlas MongoDB Cluster
Connected

:8082 API Port
Listening

JWT Auth Method
Stateless

4.1 Spring Boot
Latest

Service Status Monitor

MongoDB Connection	92%	Spring Boot API	100%
JWT Auth Service	98%	AI Crop Advisory	85%
Marketplace Module	70%	Security Filter Chain	100%

Snippet 8 CODE SECTION

Enterprise Security (JWT Authentication) demonstrates how you securely authenticate a user and generate a stateless JSON Web Token (JWT), ensuring robust Role-Based Access Control (RBAC)

```

1  /** Authenticate and return a signed JWT. */
2  @PostMapping("/login")
3  public ResponseEntity<AuthResponse> login(
4      @Valid @RequestBody LoginRequest request) {
5
6      // Throws BadCredentialsException if credentials are wrong (handled by GlobalExceptionHandler)
7      authenticationManager.authenticate(
8          new UsernamePasswordAuthenticationToken(
9              request.getEmail(),
10             request.getPassword()
11         )
12     );
13
14     // Look up user through the service layer only
15     User user = userService.findByEmail(request.getEmail());
16
17     if (user.isSuspended()) {
18         return ResponseEntity.status(403).body(new AuthResponse("Account Suspended", "NONE"));
19     }
20
21     String token = jwtService.generateToken(user.getEmail());
22
23     return ResponseEntity.ok(new AuthResponse(token, user.getRole()));
24 }
25 }

```

Generative AI Integration with Network Resilience from AIController.java proves you didn't just connect to an API, but you engineered network resilience. It shows the RestTemplate connecting to Google Gemini, with a custom retry loop to handle HTTP 503 Server Overload errors gracefully

```

1  // Retry up to 3 times to handle temporary 503 overload errors
2  int maxRetries = 3;
3  for (int attempt = 1; attempt <= maxRetries; attempt++) {
4      try {
5          ResponseEntity<Map> response = restTemplate.exchange(url, HttpMethod.POST, entity, Map.class);
6          Map<String, Object> body = response.getBody();
7
8          if (body != null && body.containsKey("candidates")) {
9              List<Map<String, Object>> candidates = (List<Map<String, Object>>) body.get("candidates");
10             if (!candidates.isEmpty()) {
11                 Map<String, Object> content = (Map<String, Object>) candidates.get(0).get("content");
12                 List<Map<String, Object>> parts = (List<Map<String, Object>>) content.get("parts");
13                 if (parts != null && !parts.isEmpty()) {
14                     return (String) parts.get(0).get("text");
15                 }
16             }
17         }
18         return "Sorry, I could not generate a response at this time. Please try again.";
19     } catch (Exception e) {
20         System.err.println("Gemini API attempt " + attempt + " failed: " + e.getMessage());
21         if (attempt < maxRetries) {
22             try { Thread.sleep(1500); } catch (InterruptedException ie) { Thread.currentThread().interrupt(); }
23         } else {
24             return "The AI is currently experiencing high demand. Please wait a few seconds and try your question again! 🤖";
25         }
26     }
27 }
28
29 return "Sorry, I could not connect to the AI at this time.";
30 }
31 }
32

```

MongoDB Persistence & Workflow Logic This snippet from `MarketplaceController.java` shows how you interact with the MongoDB Cloud Database. It demonstrates how a farmer submits a marketplace listing and how it defaults to a **PENDING** status for Admin moderation.

```

1  /** Farmer submits a new listing — saved as PENDING until Admin approves. */
2  @PostMapping("/listings")
3  public ResponseEntity<MarketplaceListing> createListing(
4      @RequestBody MarketplaceListing listing) {
5
6      listing.setId(null); // ensure Mongo generates the ID
7      listing.setStatus("PENDING"); // always starts as PENDING
8      listing.setCreatedAt(LocalDateTime.now());
9
10     if (listing.getFarmerName() == null || listing.getFarmerName().isBlank()) {
11         listing.setFarmerName("Anonymous Farmer");
12     }
13
14     MarketplaceListing saved = listingRepository.save(listing);
15     return ResponseEntity.status(201).body(saved);
16 }

```

Admin Dispute Moderation (REST API) This snippet from `AdminMarketplaceController.java` demonstrates how your platform allows Administrators to resolve marketplace disputes. It uses modern HTTP `@PatchMapping` to update specific fields without altering the whole record.

```

1  @PatchMapping("/disputes/{id}/status")
2  public ResponseEntity<Map<String, String>> updateDisputeStatus(
3      @PathVariable String id,
4      @RequestBody Map<String, String> body) {
5
6      Map<String, String> response = new HashMap<>();
7      Optional<Dispute> optDispute = disputeRepository.findById(id);
8
9      if (optDispute.isEmpty()) {
10         response.put("message", "Dispute not found");
11         return ResponseEntity.status(404).body(response);
12     }
13
14     String newStatus = body.getDefault("status", "OPEN").toUpperCase();
15     if (!List.of("OPEN", "RESOLVED", "DISMISSED").contains(newStatus)) {
16         response.put("message", "Invalid status");
17         return ResponseEntity.badRequest().body(response);
18     }
19
20     Dispute dispute = optDispute.get();
21     dispute.setStatus(newStatus);
22     disputeRepository.save(dispute);
23
24     response.put("message", "Dispute status updated to " + newStatus);
25     return ResponseEntity.ok(response);
26 }
27 }

```

Data Integrity & Exception Handling This snippet from `UserService.java` demonstrates professional exception handling during the Registration process. It ensures no two users can register with the same email, preventing database corruption.

```

1  Register a new user.
2  * @throws DataIntegrityViolationException if email already exists
3  */
4  public User registerUser(RegisterRequest request) {
5
6      // Explicit duplicate-email check with a clear error message
7      if (userRepository.findByEmail(request.getEmail()).isPresent()) {
8          throw new DataIntegrityViolationException(
9              "An account with email '" + request.getEmail() + "' already exists."
10         );
11     }
12
13     User user = User.builder()
14         .name(request.getName())
15         .email(request.getEmail())
16         .password(passwordEncoder.encode(request.getPassword()))
17         .role("USER")
18         .build();
19
20     return userRepository.save(user);
21 }

```

Spring Security Configuration (Endpoint Protection) This snippet from `SecurityConfig.java` is arguably the most important file in the backend. It proves understand how to build a `SecurityFilterChain` to protect specific endpoints (like the admin dashboard) while allowing public access to the landing page and login portals. It also explicitly configures the application to be completely STATELESS using JWTs.

```

1  @Bean
2  public AuthenticationProvider authenticationProvider() {
3      DaoAuthenticationProvider provider =
4          new DaoAuthenticationProvider(userDetailsService);
5
6      provider.setPasswordEncoder(passwordEncoder);
7
8      return provider;
9  }
10
11  @Bean
12  public AuthenticationManager authenticationManager(
13      AuthenticationConfiguration configuration) throws Exception {
14      return configuration.getAuthenticationManager();
15  }
16 }

```

Cloud NoSQL Database Architecture This snippet from User.java showcases how you transitioned away from traditional SQL databases to use a flexible MongoDB NoSQL Document structure. Notice how the Gamification fields (XP and Level) are built directly into the core User model

```
1  @Document(collection = "users")
2  @Data
3  @NoArgsConstructor
4  @AllArgsConstructor
5  @Builder
6  public class User {
7
8      @Id
9      private String id;
10
11     private String name;
12
13     @Indexed(unique = true)
14     private String email;
15
16     private String password;
17
18     /** Allowed values: USER, VET, ADMIN */
19     private String role;
20
21     // — Gamification —————
22     @Builder.Default
23     private int xp = 40;
24
25     @Builder.Default
26     private int level = 1;
27
28     private java.util.List<Boolean> missionsState;
29
30     // — Admin Control —————
31     @Builder.Default
32     private boolean suspended = false;
33
34     // — Farmer Profile Details —————
35     private String phone;
36
37     /** e.g. "Black Soil", "Red Soil", "Alluvial Soil", "Clay Soil", "Sandy Soil" */
38     private String soilType;
39
40     /** Land size in acres */
41     private Double landSize;
42
43     /** Preferred UI language: "English", "Telugu", "Hindi" */
44     @Builder.Default
45     private String language = "English";
46
47     // — Subscription —————
48     @Builder.Default
49     private String subscriptionPlan = "FREE";
50 }
```

Gamification Engine Synchronization (REST API) This snippet from `UserController.java` proves how the backend keeps track of the gamification engine. It shows how securely pull the currently authenticated user from the `SecurityContextHolder`, and update their Experience Points (XP), Level, and Daily Missions in the MongoDB database.

```
/** Safely retrieve the currently logged-in user from the JWT Security Context */
private User getAuthenticatedUser() {
    Authentication auth = SecurityContextHolder.getContext().getAuthentication();
    if (auth == null || !auth.isAuthenticated() || auth.getName().equals("anonymousUser")) {
        return null;
    }
    return userService.findByEmail(auth.getName());
}

/** POST /api/user/sync - Sync XP, level, and mission checkbox state to Database */
@PostMapping("/sync")
public ResponseEntity<ApiResponse> syncUserData(@RequestBody UserSyncDto syncDto) {

    User user = getAuthenticatedUser();

    if (user == null) {
        return ResponseEntity.status(401).body(new ApiResponse("Unauthorized"));
    }

    // Update the gamification metrics from the frontend
    user.setXp(syncDto.getXp());
    user.setLevel(syncDto.getLevel());
    user.setMissionsState(syncDto.getMissionsState());

    // Persist to MongoDB
    userService.saveUser(user);

    return ResponseEntity.ok(new ApiResponse("Data synced successfully"));
}
```

7. Challenges Faced

During the development and integration phases of the FarmVerse project, several technical and operational challenges were encountered. These were systematically addressed through engineering best practices:

1. Technical Challenge: AI API Deprecation and Rate Limiting (HTTP 503 Errors)

- **The Challenge:** While integrating the Generative AI advisory system, Google updated its AI models, deprecating the older endpoint which caused the application to throw 404 Not Found errors. Furthermore, once migrated to the newer gemini-3.6-flash model, intermittent high server demand caused the API to return 503 Service Unavailable errors, causing the chatbot to crash and fail to return advice to the farmer.
- **The Resolution:** The backend `AIController.java` was heavily refactored. The endpoint was updated to the latest model, and a robust **Fallback & Retry Mechanism** was engineered.

A network loop was implemented to automatically catch 503 errors and retry the API request up to 3 times with a 1.5-second delay. If all retries fail, it gracefully returns a user-friendly message rather than crashing the application.

2. Technical Challenge: Secure Credential Management

- **The Challenge:** Hardcoding the MongoDB database password and the Google Gemini API key directly into the application.properties file posed a critical security vulnerability, especially when committing code to version control (GitHub).
- **The Resolution:** All sensitive credentials were removed from the source code. The Spring Boot configuration was modified to dynamically inject these secrets at runtime using environment variables (e.g., \${MONGO_PASSWORD}). During execution, these variables are passed directly through the terminal command line, ensuring enterprise-grade security.

3. Operational Challenge: Ensuring High Performance for Rural Users

- **The Challenge:** Farmers in rural areas often suffer from low-bandwidth internet connections. Building a heavy frontend with large JavaScript frameworks would result in slow load times and a poor user experience.
- **The Resolution:** A strategic architectural decision was made to discard heavy frameworks (like React.js). The frontend was developed purely using native HTML5, CSS3, and Vanilla JavaScript. To keep the app responsive without reloading the page, asynchronous fetch() API calls were heavily utilized. This resulted in an ultra-lightweight, lightning-fast dashboard that consumes minimal data bandwidth.

8. Learnings & Skills Acquired

During the course of this internship and the development of the FarmVerse project, I gained substantial hands-on experience across the entire software development stack. The project served as a rigorous practical application of computer science principles, resulting in the following key takeaways:

1. Advanced Technical & Tool Skills

- **Backend Architecture (Java/Spring Boot):** Developed a deep understanding of building scalable RESTful APIs using Java 21 and the Spring Boot framework. Learned how to manage dependencies with Maven and construct a modular Controller-Service-Repository architecture.
- **Enterprise Security (JWT):** Acquired practical knowledge of implementing stateless authentication using JSON Web Tokens (JWT) and configuring Spring Security for Role-Based Access Control (RBAC).
- **Cloud Database Management (MongoDB Atlas):** Transitioned from traditional relational databases (SQL) to mastering NoSQL document storage. Learned how to connect a Spring Boot application to a cloud-hosted MongoDB cluster and perform asynchronous CRUD operations.
- **Frontend Web Development:** Mastered native web technologies (HTML5, CSS3, and Vanilla JavaScript), specifically focusing on asynchronous fetch() operations to build dynamic, single-page-like applications without relying on heavy frameworks.

2. API Integration & Cloud AI Implementation

- **Generative AI Integration:** Gained cutting-edge experience working with Large Language Models (LLMs). Successfully integrated the Google Gemini 3.6 Flash API into a Java backend using Spring's RestTemplate.
- **Network Resilience Engineering:** Learned how to handle real-world network instability and API rate limits by writing custom exponential backoff and retry logic to mitigate HTTP 503 server errors.

3. Domain Knowledge & Operational Security

- **Credential Abstraction:** Learned the critical importance of cybersecurity best practices, specifically how to abstract sensitive data (API keys, database passwords) out of source code and into system environment variables.
- **Agri-Tech Domain:** Developed a strong understanding of how software can solve socio-economic problems, specifically regarding how gamification and digital marketplaces can directly empower the agricultural sector.

4. Soft Skills & Professional Growth

- **Problem-Solving & Debugging:** Enhanced my ability to read stack traces, diagnose server crashes, and independently research and resolve API deprecation issues.
- **Agile Execution & Architecture:** Learned how to take an abstract problem statement, design a full system architecture diagram, and execute it phase-by-phase into a production-ready application.

9. Testimonials from team

Our Experience Working on the FarmVerse project as part of Team C has been an incredibly rewarding and transformative experience. From day one, our team was united by a shared vision: to build a piece of software that wasn't just technically impressive, but could actively solve real-world problems for the agricultural community.

The internship provided us with a fast-paced, collaborative environment that mirrored professional software engineering teams. We learned the true value of clear communication—especially when debugging complex integrations between the frontend dashboard and the Spring Boot backend. Whenever we encountered roadblocks, such as navigating the deprecation of AI API endpoints or securing our MongoDB database, we tackled the problems collaboratively, sharing documentation and brainstorming architectural solutions together.

Key Success Points & Highlights

- **Delivering a Full-Stack Product:** We successfully took an abstract concept and turned it into a fully functional, end-to-end web application. Seeing our Java backend and Vanilla JS frontend successfully communicate for the first time was a major milestone.
- **Mastering Generative AI Integration:** One of our proudest moments was successfully implementing the Google Gemini 3.6 Flash API. Overcoming the initial 503 Service Unavailable network errors by engineering our own custom retry-logic gave us immense confidence in our backend programming skills.
- **Implementing Gamification:** Successfully coding the Experience Points (XP) and Daily Missions engine proved our ability to think beyond basic CRUD applications and focus heavily on user psychology and retention.

- **Security Implementation:** We are highly proud of successfully securing the application. Transitioning from hardcoded data to injecting environment variables and implementing JSON Web Tokens (JWT) proved that we can write enterprise-grade, secure software.

Final Thoughts Ultimately, this project bridged the gap between academic theory and industry reality. We walk away from this internship not just as students who know how to code, but as a cohesive team of developers capable of planning, executing, and deploying modern, secure, and AI-driven software architecture.

10. Conclusion

The internship has been an exceptionally enriching and defining milestone in my academic and professional journey. The overall experience of architecting, developing, and deploying the **FarmVerse** platform allowed me to transition from understanding theoretical computer science concepts in a classroom to applying them in a high-stakes, real-world software engineering environment.

Impact of the Internship The impact of this internship was two-fold. Technically, it dramatically accelerated my proficiency as a Full-Stack Developer. I gained invaluable, hands-on experience working with industry-standard technologies—from securing Java Spring Boot APIs with JSON Web Tokens to managing cloud-based NoSQL databases via MongoDB Atlas, and successfully integrating cutting-edge Generative AI using Google Gemini. Professionally, it instilled in me an engineering mindset. I learned that software development is not just about writing code; it is about architecture, cybersecurity, error mitigation, and creating user-centric designs that solve genuine socio-economic problems.

Alignment with Academic and Career Goals This internship aligns perfectly with my long-term career goal of becoming a Senior Software Engineer specializing in scalable web applications and AI integrations. Building FarmVerse directly contributed to my academic focus on web technologies, database management, and software engineering life cycles (SDLC). The experience of leading a project that actively addresses the technological gap in the agricultural sector has also inspired me to seek out future roles where I can leverage modern software, such as cloud computing and artificial intelligence, to build products that deliver meaningful, positive impacts on society.

Ultimately, this internship provided the exact technical foundation, professional confidence, and industry exposure required to excel in the modern tech industry.

11. Acknowledgements

I would like to express my sincere gratitude to **Infosys Springboard** for providing the opportunity and resources to undertake this internship. This experience allowed me to bridge the gap between academic theory and practical software engineering.

A special thanks goes to my Project Guide, **Ragul S.** our technical expertise, patient mentorship, and valuable feedback were instrumental in the successful development of the FarmVerse platform. Thank you for guiding me through the complexities of Full-Stack development and AI integration.

Finally, I am deeply thankful to my fellow teammates in **Team C.** The collaborative spirit, shared dedication, and late-night debugging sessions made this journey incredibly rewarding. Turning the FarmVerse concept into a fully functional reality would not have been possible without our collective teamwork.